

Deep Learning

Introduction to scikit-learn

Core API, Classification, Regression, Preprocessing, CV, and
Hyperparameter Search

G. Marques

What is scikit-learn?

- Python library for classical Machine Learning
- Built on NumPy, SciPy, matplotlib
- Unified API across estimators
- Supports:
 - ▶ Classification, regression, clustering
 - ▶ Dimensionality reduction
 - ▶ Model selection (CV, hyperparameter search)
 - ▶ Preprocessing + pipelines

Basic Classification Workflow

General Pattern:

- 1 Split data (train/test) or use cross-validation
- 2 Instantiate model
- 3 `fit(X_train, y_train)`
- 4 `predict(X_test)` or `predict_proba(X_test)`
- 5 Evaluate (accuracy, confusion matrix, ROC/PR, etc.)

Core Methods: `fit`, `predict`, `score`

Working Example Dataset

Wisconsin Breast Cancer (Diagnostic) dataset

- Binary classification: benign vs malignant
- 30 numeric features (computed from digitized images)
- Included in scikit-learn:

```
sklearn.datasets.load_breast_cancer
```

Typical goal: maximize detection of malignant cases (recall), while controlling false positives.

Binary Classification: Train + Predict + Score (Breast Cancer)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

pipe = Pipeline([("scaler", StandardScaler()),
                  ("clf", LogisticRegression(max_iter=2000))])
pipe.fit(X_train, y_train)
acc = pipe.score(X_test, y_test)
y_pred = pipe.predict(X_test)
print("Test accuracy:", acc)
```

Binary Classification: Thresholding

In binary classification, you often want to change the decision threshold:

$$\hat{y} = \begin{cases} 1 & p \geq \tau \\ 0 & p < \tau \end{cases}$$

- Default is usually $\tau = 0.5$
- Lower $\tau \Rightarrow$ higher recall (fewer false negatives), more false positives

Useful outputs:

- `predict_proba(X)` gives probabilities (when supported)
- `decision_function(X)` gives real-valued scores (e.g., SVM, logistic regression)

Adjusting Threshold (Breast Cancer)

```
import numpy as np
from sklearn.metrics import confusion_matrix

probs = pipe.predict_proba(X_test)[: , 1]    # P(class=1)
tau = 0.30
y_pred_tau = (probs >= tau).astype(int)

cm = confusion_matrix(y_test, y_pred_tau)
print("Threshold:", tau)
print(cm)
```

Evaluation: Confusion Matrix + Common Metrics

Binary confusion matrix:

	$\hat{y} = 0$	$\hat{y} = 1$
$y = 0$	TN	FP
$y = 1$	FN	TP

Metrics:

- Accuracy: overall correctness
- Precision: $\frac{TP}{TP+FP}$ (how many predicted positives are correct)
- Recall (TPR): $\frac{TP}{TP+FN}$ (how many actual positives found)
- F1: harmonic mean of precision and recall

Binary Evaluation (Breast Cancer)

```
from sklearn.metrics import (confusion_matrix, accuracy_score,  
                             precision_score, recall_score, f1_score)  
  
y_pred = pipe.predict(X_test)  
  
print("Accuracy :", accuracy_score(y_test, y_pred))  
print("Precision:", precision_score(y_test, y_pred))  
print("Recall    :", recall_score(y_test, y_pred))  
print("F1       :", f1_score(y_test, y_pred))  
print("CM:\n", confusion_matrix(y_test, y_pred))
```

ROC and Precision-Recall Curves (Binary)

ROC: TPR vs FPR as threshold varies. AUC summarizes curve.

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

Precision-Recall: precision vs recall as threshold varies.

- Often more informative under class imbalance

ROC / PR Curves (Breast Cancer)

```
from sklearn.metrics import roc_curve, auc, precision_recall_curve

probs = pipe.predict_proba(X_test)[: , 1]

fpr, tpr, _ = roc_curve(y_test, probs)
roc_auc = auc(fpr, tpr)

precision, recall, _ = precision_recall_curve(y_test, probs)

print("ROC AUC:", roc_auc)
```

Multi-class Evaluation

For multi-class:

- Confusion matrix becomes $K \times K$
- Accuracy still valid but can hide per-class performance
- Use per-class metrics + averaging:
 - ▶ **macro avg**: unweighted average across classes
 - ▶ **weighted avg**: weighted by class support
 - ▶ **micro avg**: global TP/FP/FN (often dominated by frequent classes)

Multi-class Example: Iris + Classification Report

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

clf = LogisticRegression(max_iter=2000)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Regression Workflow

Same API, different targets:

$$y \in \mathbb{R}$$

- `fit(X, y)`
- `predict(X)`
- `score(X, y)` (often R^2 for regressors)

Regression: Train + Evaluate

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score

X, y = load_diabetes(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

reg = Ridge(alpha=1.0)
reg.fit(X_train, y_train)

y_pred = reg.predict(X_test)

print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 :", r2_score(y_test, y_pred))
```

Regression Metrics

Common metrics:

- MSE / RMSE
- MAE
- R^2 (explained variance)

$$R^2 \in (-\infty, 1]$$

Preprocessing: Standardization and PCA

Standardization (StandardScaler)

$$x' = \frac{x - \mu}{\sigma}$$

Helps models sensitive to feature scale (logistic regression, SVM, kNN, PCA).

PCA

- Reduce dimensionality
- Decorrelate features
- Often improves speed + generalization (depending on model)

StandardScaler + PCA (Typical Pattern)

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

pca = PCA(n_components=2)
X_train_p = pca.fit_transform(X_train_s)
X_test_p = pca.transform(X_test_s)

print("Explained variance ratio:", pca.explained_variance_ratio_)
```

Pipelines (Avoid Data Leakage)

Best practice: wrap preprocessing + model in a Pipeline.

- Ensures transforms are fit only on training folds
- Cleaner code
- Essential for cross-validation + hyperparameter search

Pipeline Example (Scaler + Logistic Regression)

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=2000))
])
```

Why Cross-validation?

Train/test split gives one estimate; it may be unstable.

Cross-validation (CV):

- Repeatedly splits data into train/validation
- Reports mean and variability of performance
- Better model selection, especially with limited data

Common CV Strategies

- **K-Fold**: split into K folds, rotate validation fold
- **StratifiedKFold**: preserves class proportions (classification)
- **ShuffleSplit**: repeated random splits

ShuffleSplit is useful when:

- You want many random train/test partitions
- Dataset is large enough that random splits are stable

cross_val_score (Breast Cancer + Pipeline)

```
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)

scores = cross_val_score(pipe, X, y, cv=5, scoring="accuracy")

print("CV mean:", scores.mean())
print("CV std :", scores.std())
```

ShuffleSplit (Repeated Random Splits)

```
from sklearn.model_selection import ShuffleSplit, cross_val_score

cv = ShuffleSplit(n_splits=20, test_size=0.2, random_state=42)

scores = cross_val_score(pipe, X, y, cv=cv, scoring="accuracy")

print("ShuffleSplit mean:", scores.mean())
print("ShuffleSplit std :", scores.std())
```

cross_validate for Multiple Metrics

`cross_val_score` returns one metric.

`cross_validate` can return multiple metrics and timing:

- `return_train_score=True` to inspect overfitting
- useful with precision/recall/F1

cross_validate (Accuracy + Precision + Recall)

```
from sklearn.model_selection import cross_validate

scoring = ["accuracy", "precision", "recall", "f1"]

out = cross_validate(
    pipe, X, y, cv=5, scoring=scoring, return_train_score=False
)

for k in scoring:
    print(k, out["test_" + k].mean())
```

Hyperparameters vs Parameters

Parameters: learned from data (e.g., weights).

Hyperparameters: set before training (e.g., regularization strength).

Goal: select hyperparameters that generalize well using CV.

GridSearchCV vs RandomizedSearchCV

GridSearchCV

- Exhaustive search over a specified grid
- Good for small grids

RandomizedSearchCV

- Samples combinations randomly
- Scales better to large spaces
- Often finds strong configs faster

GridSearchCV (Breast Cancer + Pipeline)

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {  
    "clf__C": [0.01, 0.1, 1.0, 10.0, 100.0],  
    "clf__penalty": ["l2"],  
    "clf__solver": ["lbfgs"]  
}
```

```
grid = GridSearchCV(  
    estimator=pipe,  
    param_grid=param_grid,  
    cv=5,  
    scoring="f1",  
    n_jobs=-1)  
grid.fit(X, y)
```

```
print("Best params:", grid.best_params_)  
print("Best CV score:", grid.best_score_)
```

RandomizedSearchCV (Example)

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform
```

```
param_dist = {"clf__C": loguniform(1e-3, 1e3)}
```

```
rand = RandomizedSearchCV(
    estimator=pipe,
    param_distributions=param_dist,
    n_iter=30,
    cv=5,
    scoring="f1",
    random_state=42,
    n_jobs=-1
)
```

```
rand.fit(X, y)
```

```
print("Best params:", rand.best_params_)
print("Best CV score:", rand.best_score_)
```

Notes on Hyperparameter Search

Good practice:

- Search with a **Pipeline** (prevents leakage)
- Pick metric aligned with the objective (e.g., recall for medical screening)
- Report mean CV score + variability
- After choosing hyperparameters, refit on full training set and evaluate on a final test set

Summary

- Classification: `fit`, `predict`, `score`
- Binary: adjust threshold via `predict_proba` / `decision_function`
- Evaluation: confusion matrix; ROC/PR curves for binary
- Multi-class: confusion matrix + macro/micro/weighted averaging
- Cross-validation: `KFold`/`StratifiedKFold`, `ShuffleSplit`, `cross_validate`
- Hyperparameter search: `GridSearchCV`, `RandomizedSearchCV`